

C PROGRAMMING CONTROL FLOW

Mastering Loops in C

Comprehensive guide to iteration using `while`, `for`, and `do-while` constructs with practical code programs.

— Programming Fundamentals Series

Why Do We Need Loops?

CORE BENEFIT

90%

Code line reduction vs manual repetition.



Automated Repetition

Executes task blocks dynamically.



Dynamic Control

Terminates on flexible runtime conditions.



Memory Efficiency

Optimizes execution and binary footprint.

3 Structural Pillars of Any Loop

Every iterative construct in C relies on three mandatory control steps.

STEP 1

Initialization

Sets starting counter variable before loop execution.

STEP 2

Condition Check

Evaluates boolean expression before each iteration.

STEP 3

Update Expression

Modifies counter value to ensure eventual termination.

The while Loop: Entry-Controlled Iteration

```
// Program: Countdown from 5 to 1
#include

int main() {
    int count = 5; // 1. Initialization

    // 2. Condition evaluated before entry
    while (count > 0) {
        printf("%d ", count);
        count--; // 3. Decrement update
    }
    return 0;
}
```

SYNTAX RULE

Spreads setup across three separate locations.

Pre-Condition Test: Evaluates before loop body runs.

Dynamic Steps: Ideal when total iterations are unknown.

Execution Output: 5 4 3 2 1

The for Loop: Compact & Structured

```
// Program: Count 1 to 5 using for loop
#include

int main() {
    // Init, condition & update in 1 line
    for (int i = 1; i <= 5; i++) {
        printf("%d ", i);
    }
    return 0;
}
```

SYNTAX CONVENIENCE

Consolidates all control logic into loop header.

Known Iterations: Best when exact count is predetermined.

Variable Scope: Counter localized directly inside loop.

Execution Output: 1 2 3 4 5

Program: Summing First N Numbers

Comparing implementation logic between **while** and **for** loops for N = 10.

APPROACH 1: WHILE LOOP

```
int i = 1, sum = 0;

while (i <= 10) {
    sum += i;
    i++;
}
printf("Sum = %d\n", sum);
// Output: Sum = 55
```

APPROACH 2: FOR LOOP

```
int sum = 0;

for (int i = 1; i <= 10; i++) {
    sum += i;
}

printf("Sum = %d\n", sum);
// Output: Sum = 55
```

Program: Multiplication Table Generator

```
// Print multiplication table of 5
#include

int main() {
    int num = 5;
    for (int i = 1; i <= 5; i++) {
        printf("%d x %d = %d\n",
               num, i, num * i);
    }
    return 0;
}
```

TERMINAL OUTPUT

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
```

Infinite Loops & Safeguards

Understanding unintended freezing and intentional loop control statements.

⚠️ Forgotten Update

Omitting increment or decrement causes infinite loop execution.

∞ Always-True Condition

Constructs like `while(1)` run endlessly until forcefully stopped.

👋 break Statement

Exits loop structure immediately regardless of condition state.

▶▶ continue Statement

Skips remaining body code and advances to next iteration.

Loop Comparison Matrix

Feature	for Loop	while Loop	do-while Loop
Condition Check	At Start (Entry-Controlled)	At Start (Entry-Controlled)	At End (Exit-Controlled)
Min Executions	0 Times	0 Times	1 Time (Guaranteed)
Primary Use Case	Known iteration count	Unknown iteration count	Menu prompts & input validation



Do-While Special Rule: Notice the trailing semicolon syntax: `do { ... } while (condition);`

SUMMARY & BEST PRACTICES

Control Flow Mastered

1. Choose Wisely

Use for loops for numeric ranges; use while loops for conditional events.

2. Guard Updates

Ensure counter variables always progress toward termination.

3. Clean Scope

Keep loop bodies concise and minimize unnecessary variable nesting.